If having multiple threads access data from the same cache line is bad, how does the memory layout of data accessed by a single thread affect things?

8.2.4. How close is your data?

Whereas false sharing is caused by having data accessed by one thread too close to data accessed by another thread, another pitfall associated with data layout directly impacts the performance of a single thread on its own. The issue is data proximity: if the data accessed by a single thread is spread out in memory, it's likely that it lies on separate cache lines. On the flip side, if the data accessed by a single thread is close together in memory, it's more likely to lie on the same cache line. Consequently, if data is spread out, more cache lines must be loaded from memory onto the processor cache, which can increase memory access latency and reduce performance compared to data that's located close together.

Also, if the data is spread out, there's an increased chance that a given cache line containing data for the current thread also contains data that's *not* for the current thread. At the extreme there'll be more data in the cache that you don't care about than data that you do. This wastes precious cache space and thus increases the chance that the processor will experience a cache miss and have to fetch a data item from main memory even if it once held it in the cache, because it had to remove the item from the cache to make room for another.

Now, this is important with single-threaded code, so why am I bringing it up here? The reason is *task switching*. If there are more threads than cores in the system, each core is going to be running multiple threads. This increases the pressure on the cache, as you try to ensure that different threads are accessing different cache lines in order to avoid false sharing. Consequently, when the processor switches threads, it's more likely to have to reload the cache lines if each thread uses data spread across multiple cache lines than if each thread's data is close together in the same cache line.